There's one final GenServer callback that's important to know about. Modify the existing init/1 function:

```
def init(speaker: speaker) do
  :timer.send_interval(5000, :announce)
  {:ok, %{speaker: speaker, last_result: nil}}
end
```

And add a corresponding handle_info/2 at the bottom of the file:

```
def handle_info(
  :announce,
  state = %{speaker: speaker, last_result: result}
) do
  Logger.info("#{speaker}: The last result I computed was #{result}")
  {:noreply, state}
end
```

handle_info/2 looks similar to handle_cast/2, but it's used for non-GenServer messages. For example, if you send a message to this process, then it's handled by the handle_info/2 callback. We use the :timer.interval/2 function to automatically send a message to the process every five seconds.

When you start this process, you'll see a message every five seconds:

```
$ iex -S mix
iex> alias Examples.GenServer.SimpleServer
iex> {:ok, pid} = SimpleServer.start_link([speaker: "Genny"], name: nil)
{:ok, #PID<0.149.0>}

iex> SimpleServer.add(5, 10, pid)
15
[info] Genny: The last result I computed was 15

iex(5)> SimpleServer.add(50, 70, pid)
120
[info] Genny: The last result I computed was 120
[info] Genny: The last result I computed was 120
...continues forever
```

The GenServer process will log out its last result forever because the process runs until it's explicitly exited. We won't get into the details of all of the ways that a process can exit, but you do have full control of how a process terminates.

Is a GenServer an Object?

A common reaction when seeing GenServer is to think "oh, that's an object!" Objects and GenServers have a lot in common. They both hold state, the modifications to the state are colocated in code, they receive messages (at